Correction TP4

1- Gestion des utilisateurs

a) Créer une nouvelle base de données « testbdd » et créer un nouveau rôle ayant tous les privilèges sur cette base, mais pas les droits de création d'une nouvelle base de données.

```
sudo -u postgres psql
create database testbdd
create role tesbddr inherit login password 'test';
\c testbdd
grant all privileges on all tables in schema public to testbddr;
\q
psql testbdd tesbddr
```

b) Créer 2 tables dans votre base (« table1 » et « table2 ») avec les champs que vous souhaitez.

```
create table table1 (id serial, value integer, primary key(id));

create table table2 (id2 serial, value2 integer, primary key(id2));
```

c) A partir du premier rôle créé en a), créer un deuxième utilisateur (rôle) ayant accès à la table « table1 » uniquement avec les privilèges « insert », « select », « update » et « delete »

```
sudo -u postgres psql
alter role tesbddr createrole;
psql testbdd tesbddr
create role tesbddr2 inherit login password 'test2';
grant insert, select, update, delete on table table1 to tesbddr2;
psql testbdd tesbddr2
select * from table1;
select * from table2;
```

2- Gestion des vues

Dans la base de données du TP1 regroupant les tables « employe » et « service »

a) Créer une vue permettant de visualiser le nom, la fonction, le salaire + la commission, le nom du supérieur hiérarchique, le nom du service et la ville des employés.

```
create view nouvelle_vue as
         select e1.nom, e1.fonction,
         e1.salaire + coalesce(e1.comm, 0) as "revenu total",
         e2.nom as "Nom supérieur", s.type_service, s.ville
         from employe e1, employe e2, service s
         where e1.num_service = s.num_service
         and e1.numemp_sup = e2.numemp;
```

b) Construire une requête sur cette vue permettant d'afficher uniquement les employés du service « siege ».

```
select * from nouvelle_vue where type_service='siege';
```

c) Créer un nouvel utilisateur n'ayant accès qu'à cette vue (et vérifier !!).

```
create role vue_role inherit login password 'passwd';
grant all privileges on nouvelle_vue to vue_role;
psql dbtp1 vue_role
select * from employe;
select * from nouvelle_vue;
```

3- Gestion des triggers

Sur la table « employe »

a) Créer un champ « salaireetcom » de type double

```
alter table employe add salaireetcomm Numeric(7,2);
```

b) Créer un déclencheur levant une alerte si la somme de salaire et commission indiquée dans « salaireetcom » est inférieure au montant du salaire

```
create function verif_salcom() returns trigger as $verif_salcom$
begin
if new.salaireetcomm < new.salaire then
raise exception '% ne peut pas avoir de commission de valeur
négative!', new.nom;
end if;
end;
$verif_salcom$ language plpgsql;

create trigger verif_salcom before insert or update on employe for
each row execute procedure verif_salcom();


insert into employe values
(1002,'Leroux','ingenieur',1000,current_date,1900,200,40,1830);
```

4- Gestion des indexes

Dans la base de données « Sakila »,

a) Construire la requête qui affiche la durée moyenne des locations par catégorie de films. Le résultat sera présenté dans l'ordre décroissant des durées de location moyenne par catégorie.

```
select c.name, round(avg(f.rental_duration),2)
        from category c left join film_category fc
        on c.category_id=fc.category_id
        left join film f on f.film_id=fc.film_id
        group by c.name order by round(avg(f.rental_duration),2)
        desc;
```

b) Utiliser « EXPLAIN ANALYZE » avant le « SELECT » pour visualiser la réalisation de la requête et mesurer le temps d'exécution.

```
explain analyze select c.name, round(avg(f.rental_duration),2)
        from category c left join film_category fc
        on c.category_id=fc.category_id
        left join film f on f.film_id=fc.film_id
        group by c.name order by round(avg(f.rental_duration),2)
        desc;
```

c) Créer un index sur le champ « rental_duration » de la table « film », refaire la requête précédente et comparer le temps (en utilisant « EXPLAIN ANALYZE »).

```
create index ind_rental on film(rental_duration);
```

d) Déterminer la taille des tables et indexes.

```
select pg_size_pretty(pg_relation_size('film'));
```

```
select pg_size_pretty(pg_relation_size('ind_rental'));
```